
Model coherency checking tool

— What and how about this
extension of pyMaBoSS —

Table of contents

- Who am I ?
- Presentation of the tool
- Some examples
- AI usage and comparison

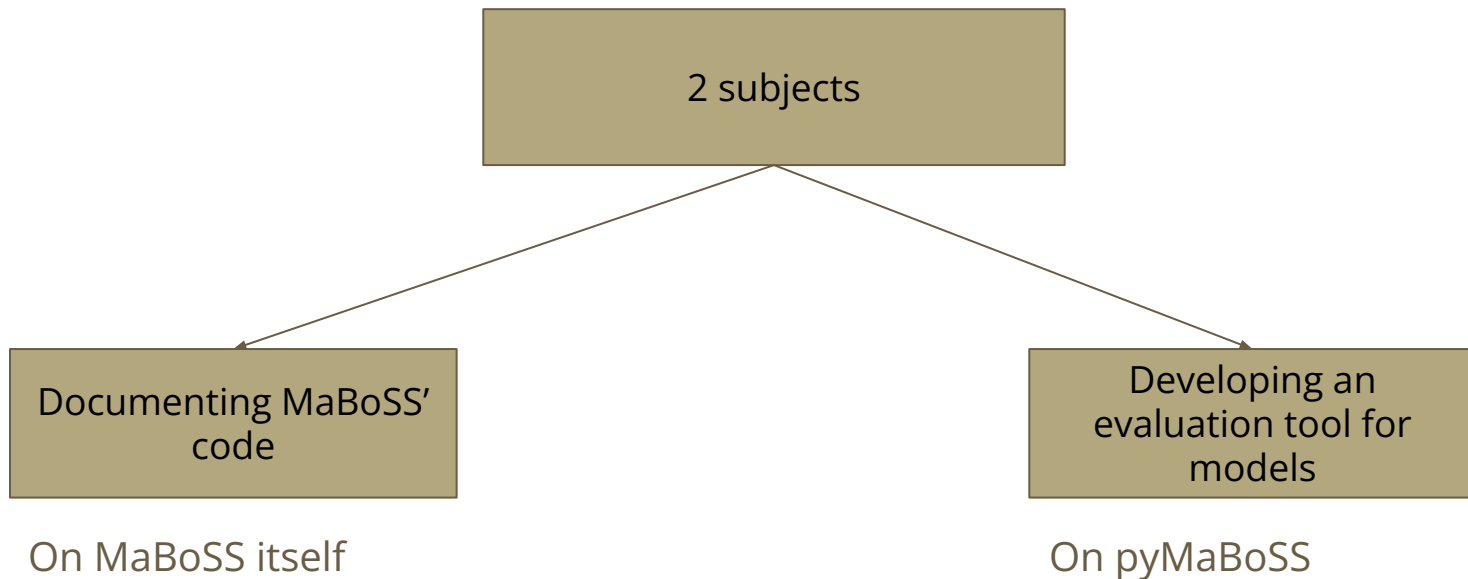
My background

- Had my baccalaureat in 2018
 - Scientific - Engineering Sciences
- BAC+3 in Industrial Computer Science, Automation and Electricity in apprenticeship
- Re-orientation in Computer Sciences in 2024
- Intern of BUT2 in Computer Science (Informatique)
- Supervised by Laurence and Gautier



Belfort, IUT de Belfort-Montbéliard

What am I doing here ?



What MaBoSS does

- Intra-cellular modelisation tool
- Understand and predict cell's behaviour following a mutation or a treatment

However:

Model construction is complex

We need a way to be sure that, for any changes in the model, it stills reproduce what it has to.

MaBoSS_test establishes a list of constraints that the model has to fulfill.

Evolution of the module

Verify that any addition to a model, does not interfere with its global coherency

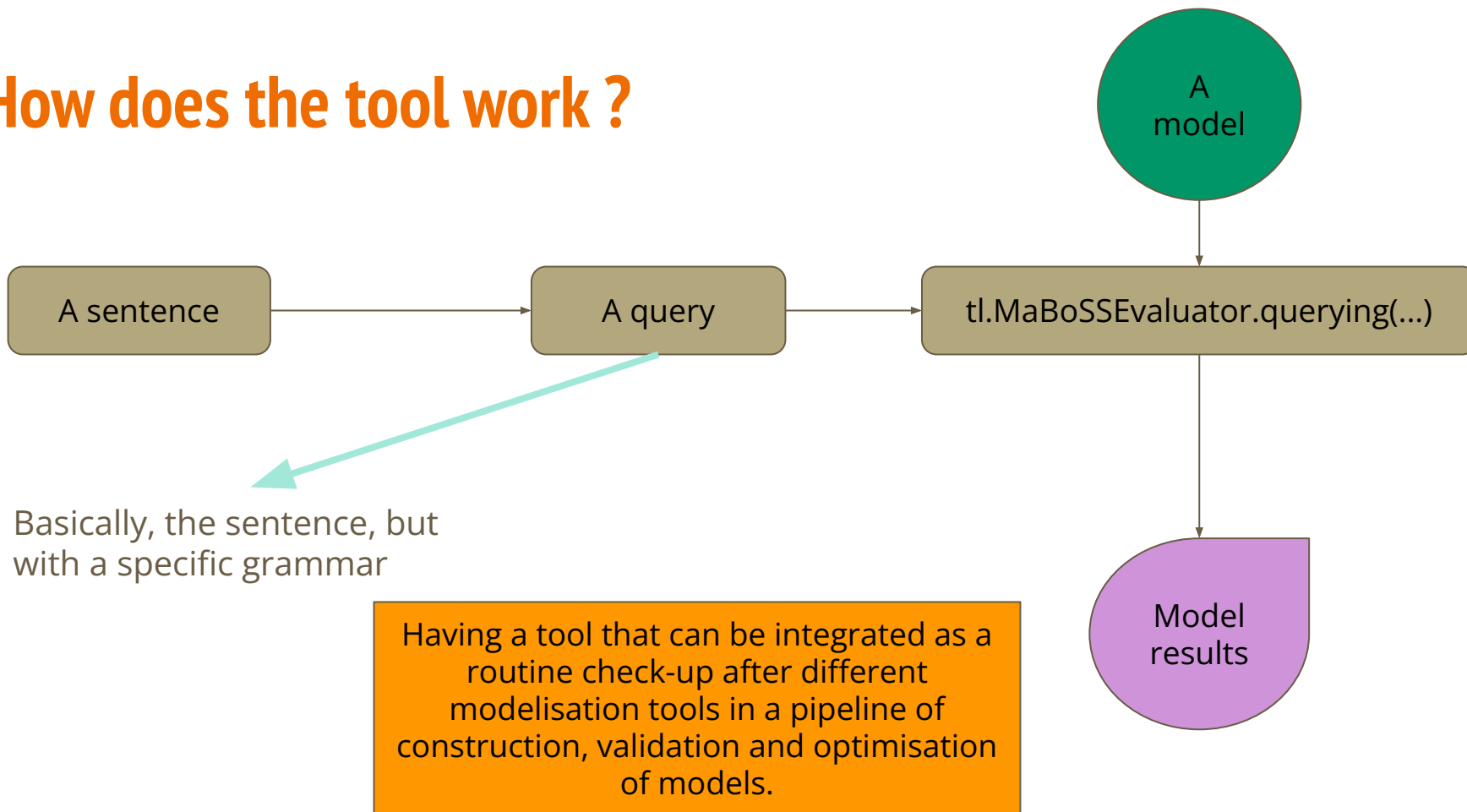
When I arrived

- MaBoSS_test : used but not published
- Hard to understand
- Code hard if not impossible to automate
- Outputs without details

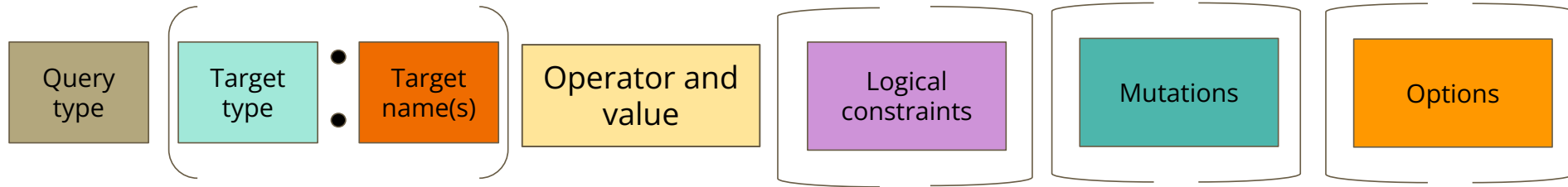
Now

- Temporal Logic of pyMaBoSS
- Strict grammar but easier to understand
- Thought to be automated
- Detailed outputs and clear error messages

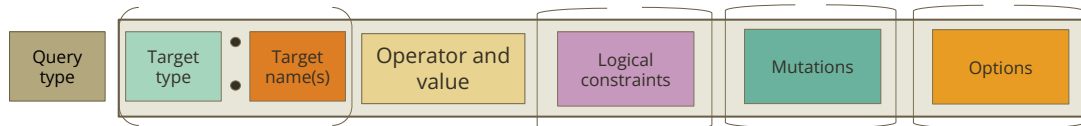
How does the tool work ?



The grammar



MEK inhibition stops ERK signal.



The grammar - Query Types

The query type determines what we want to check

→ A probability

→ An evolution

→ An interval

→ A difference

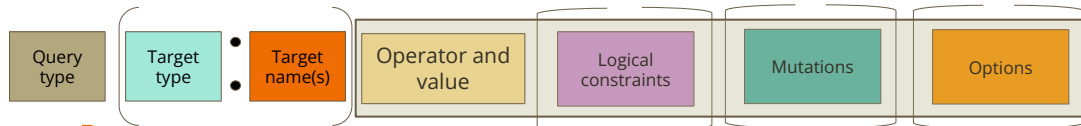
Dec(

Choosing the query type:

- Looking for an evolution:
 - Increase in probability: Inc
 - Decrease in probability: Dec
- Looking for certain probabilities:
 - Compare the probability: P
 - See probability max/min: Pmax / Pmin
- Looking for intervals:
 - All the time codes: T
 - The first time: Tmin
 - The last time: Tmax

Some key words

- **Increase:** augmented, increase in ... , activation ...
- **Decrease:** mediates resistance, decrease in...



Grammar - Target types and name

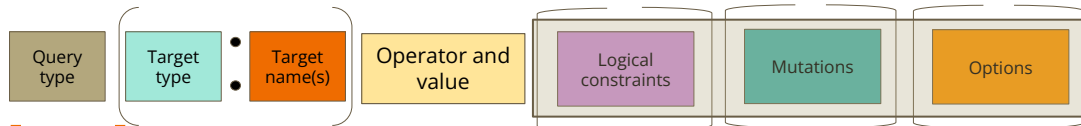
Target type is indicating what is the target to check on:

- Node
- State
- Fixpoints

Writing target type and names:

- Node : node
- State : state
- Fixpoints : fp
- Multiple names :
 - name1,name2 ...
- No spaces in the names :
 - A-B-C
- No quotes
- No brackets, no parenthesis
- Exact names are required

Dec(node:ERK_b1,ERK_b2)



Grammar - Operators and value

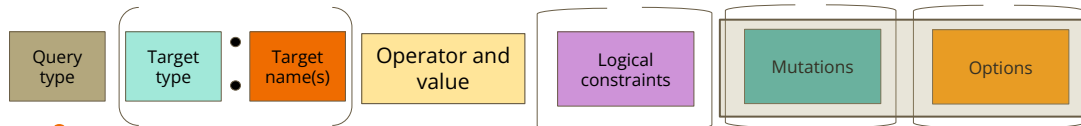
- Operators :
 - $> < >= <=$
- Only in P, T, Tmax, Tmin, Pmax, Pmin operation
- For Inc and Dec : /

Some short examples:

- $P(t:n) \geq 0.2$
- $Tmin(t:n,n2) \leq 0.1$
- $Inc(t:n,n1) /$

- Values are **probabilities** [0,1]
- No value for Inc and Dec
- $P(target:name) = ?$
 - With logical constraints

`Dec(node:ERK_b1,ERK_b2) /`



Grammar - Logical constraints

- Add additional constraints
 - Handle states and nodes
 - Allow multiple levels
-
- Handled symbols:
 - & : and
 - | : or
 - ! : not
 - All numerical comparators

`Dec(node:ERK_b1,ERK_b2) / [ ]`

How to write one constraint:

It is a combination of nodes that are active or inactive : **A & !B or node:A & node:!B**

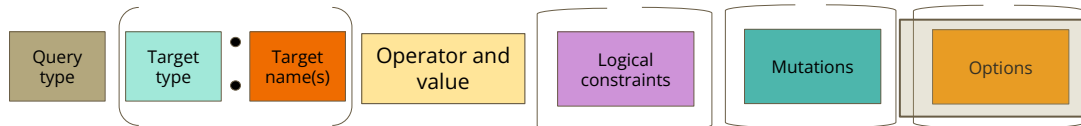
You can evaluate based on a state :
state:A | state:!B

You can have multiple levels like so :
(A | (B & (C | D)))

Spaces between each member, even parenthesis

In some cases, you can evaluate based on the probability of the node or state : **B < 0.5**

- Always **name operator value**



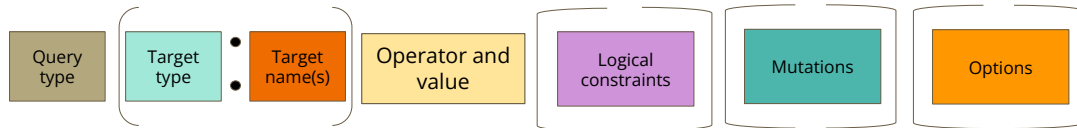
Grammar - Mutations

- Creates a different simulation with the mutations passed in the second brackets
- **Required** in Increase and Decrease operations
- An unknown node will be ignored
 - there is a message but no interruption
- No number limit

Writing mutations:

- Space between each couple node/state
 - BRAF:ON AKT:OFF
- No quotes, no spaces in the names
- Space between the bracket and the couple :
 - [BRAF:ON AKT:OFF]

```
Dec(node:ERK_b1,ERK_b2) / [ ] [ MEK_b1:OFF MEK_b2:OFF ]
```



Grammar - Options

- Make the results more precise
- Add threshold constraint
- Take into account early variations that are not visible in the end

Examples:

- Minimum percentage difference
 - *val*%
 - An integer
- Use a restricted amount of digits after the dots: (default is 4)
 - *digits:val*
 - An integer

How to write some options:

Take a variation into account:

- *transient*

4 sub options :

- *threshold:val*
 - general minimal change value
- *end:val*
 - tolerated difference between the two final values
- *start:val*
 - tolerated difference between the two initial values
- *optimum:val*
 - tolerated difference between the two min/max values

[10% digits:3 transient:threshold:0.05,start:0.04,optimum:0.04,end:0.3]

Order non-relevant

Dec(node:ERK_b1,ERK_b2) / [] [MEK_b1:OFF MEK_b2:OFF] [5% digits:3]

Examples of queries

MEK inhibition stops ERK signal.

```
Dec(node:ERK_b1,ERK_b2) / [ ] [ MEK_b1:OFF MEK_b2:OFF ] [ 5% digits:3 ]
```

Early resistance to BRAF inhibition may be observed in case of PTEN loss.

```
Inc(node:Proliferation_b1,Proliferation_b2) / [ ] [ BRAF:OFF PTEN:OFF ] [ transient:start:0.05,end:0.1,optimum:0.5 digits:3 ]
```

The results of the queries

- Presented as dataframes
- Display method to have a clean look

```
Query 1 : Dec(node:ERK_b1,ERK_b2) / [ ] [ MEK_b1:OFF MEK_b2:OFF ] [ 5% digits:3 ]
```

	ERK_b1 from master	ERK_b1 from mutation	Difference ERK_b1	Percentage ERK_b1	Decrease ERK_b1	ERK_b2 from master	ERK_b2 from mutation	Difference ERK_b2	Percentage ERK_b2	Decrease ERK_b2
0	0.476	0.0	-0.476	-100.00%	True	0.157	0.0	-0.157	-100.00%	True

val_mutation - val_master

Can we use LLMs to
automatize the writing
of the queries ?

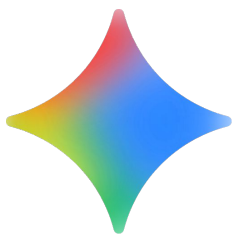
AI usage and comparison

Objective :

1. Automate the writing of queries
2. Automate the evaluations

Methodology:

Comparison of three agents on the same assertions using the exact same prompt.



The prompt

- The regular expression
- The model and comparison rules
- The syntax rules
- The output instructions

Globally, every rule that is needed to write grammatically correct queries.

Role: You are an expert compiler for the MaBoSS temporal_logic module. Your task is to translate biological assertions into Formula objects using a strict textual syntax.

1. THE VALIDATION REGEX (Mandatory): Every query must strictly match this REGEX: `^(Pmax|Pmin|P|T|Tmin|Tmax|D|M|Inc|Dec)\((node|state|fp)\((.+?)\)\)`
`(?:\s*(<=>|<|>|>=|!=|/)\s*(0(?:\s*\.\d+)?|1(?:\s*\.\d+)?|\d+)?(?:\s*\[([.*?])\]|(?:\s*\[([.*?])\])?(?:\s*\[([.*?])\])?(?:\s*\[([.*?])\])?)?)?)`

2. MODEL & COMPARISON RULES:

Model Isolation: Each query applies to a specific model (e.g., Colorectal or Melanoma). If a sentence compares two models, generate two separate queries and specify which model each belongs to.

compare: Scope: The compare: MUTATION option is used only to compare two mutational states within the same model. It cannot compare different model types.

3. SYNTAX RULES (Based on Formulas.py):

Spacing: One space on each side of the slash /. One space between each set of brackets [].

The 3 Blocks: Structure must be [LOGIC] [MUTATIONS] [OPTIONS].

Mutations & Options: Use SPACES only as separators. Commas are strictly forbidden inside brackets.

Unique Exception: Commas are allowed only to list targets node:A,B or comparison references compare:M1,M2.

Values:

? is reserved exclusively for the P type with the = operator.

Probabilities must be between 0 and 1.

Inc and Dec must use the slash / if no threshold value is provided.

4. REFERENCE EXAMPLES (Do NOT rewrite these in your output):

Simple Inhibition: `Inc(node:EGFR) / [ ] [ BRAF:OFF ] [ 5% ]`

Probability Request: `P(node:ERK) = ? [ ERK & AKT ] [ ] [ ]`

Synergy on a Model: `Dec(node:ERK) / [ ] [ BRAF:OFF MEK:OFF ] [ compare:BRAF:OFF ]`

OUTPUT INSTRUCTIONS:

Identify the required model(s).

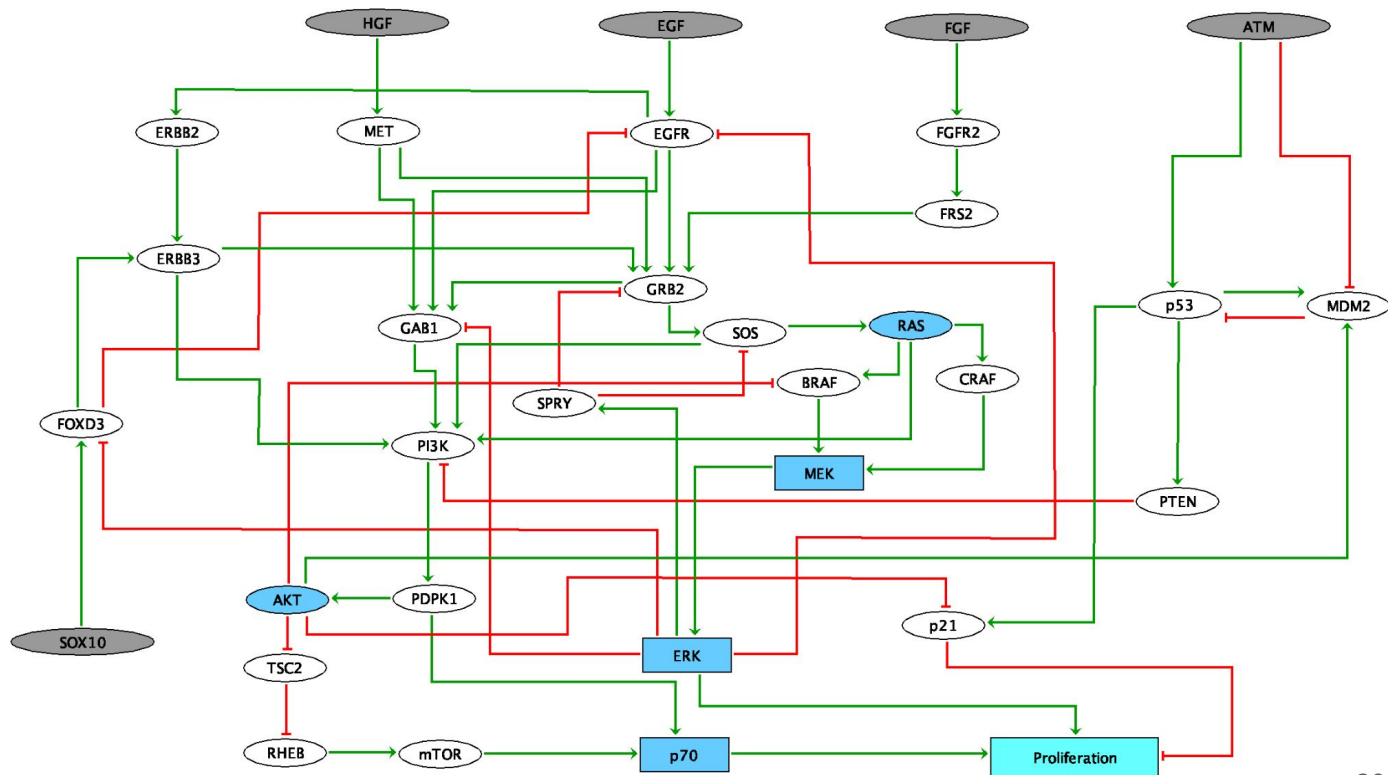
Generate the query/queries in the following format: `Model [Name]: QUERY.`

Do not rewrite the examples provided above.

Provide only the final code/result without any additional commentary.

Model studied: BRAF model

Investigates cancer response to BRAF treatments in melanomas and colorectal cancers



First assertion - ChatGPT

BRAF inhibition causes a feedback activation of EGFR in colorectal cancer and not in melanoma.

```
assertion_1 = f"Inc(node:EGFR) / [ ] [ BRAF:OFF ] [ 5% digits:3 ]"
```

Solution

```
Modèle [Colorectal] : Inc(node:EGFR) / [ ] [ BRAF:OFF ] [ ]
```

```
Modèle [Melanoma] : Dec(node:EGFR) / [ ] [ BRAF:OFF ] [ ]
```

ChatGPT

First assertion - Claude

BRAF inhibition causes a feedback activation of EGFR in colorectal cancer and not in melanoma.

```
assertion_1 = f"Inc(node:EGFR) / [ ] [ BRAF:OFF ] [ 5% digits:3 ]"
```

Solution

BRAF inhibition causes a feedback activation of EGFR in colorectal cancer and not in melanoma.

Modèle [Colorectal]: `Dec(node:EGFR) / [ ] [ BRAF:OFF ] [ compare:BRAF:OFF ]`

Modèle [Melanoma]: `Dec(node:EGFR) / [ ] [ BRAF:OFF ] [ ]`

correction : pas de compare dans le modèle colorectal

Modèle [Colorectal]: `Inc(node:EGFR) / [ ] [ BRAF:OFF ] [ ]`

Modèle [Melanoma]: `Dec(node:EGFR) / [ ] [ BRAF:OFF ] [ ]`

First assertion - Gemini

BRAF inhibition causes a feedback activation of EGFR in colorectal cancer and not in melanoma.

```
assertion_1 = f"Inc(node:EGFR) / [ ] [ BRAF:OFF ] [ 5% digits:3 ]"
```

Solution

Modèle [Colorectal] : Inc(node:EGFR) / [] [BRAF:OFF] []

Modèle [Melanome] : P(node:EGFR) / [] [BRAF:OFF] []

Gemini (fast)



Modèle [Colorectal] : Inc(node:EGFR) / [] [BRAF:OFF] []

Modèle [Mélanome] : Dec(node:EGFR) / [] [BRAF:OFF] []

Gemini (reasoning)

Second assertion - ChatGPT

Overexpression/mutation of CRAF results in constitutive activation of ERK and MEK also in the presence of a BRAF inhibitor.

```
assertion_5_a = "Inc(node:ERK,MEK_b1) / [ ] [ CRAF:ON BRAF:OFF ] [ 5% ]"  
assertion_5_b = "Inc(node:ERK,MEK_b1) / [ ] [ CRAF:ON ] [ 5% ]" #without BRAF inhibitor
```

Solution

```
Modèle [Generic] : Inc(node:ERK,MEK) / [ ] [ CRAF:ON BRAF:OFF ] [ ]
```

ChatGPT

Second assertion - Claude

Overexpression/mutation of CRAF results in constitutive activation of ERK and MEK also in the presence of a BRAF inhibitor.

```
assertion_5_a = "Inc(node:ERK,MEK_b1) / [ ] [ CRAF:ON BRAF:OFF ] [ 5% ]"  
assertion_5_b = "Inc(node:ERK,MEK_b1) / [ ] [ CRAF:ON ] [ 5% ]" #without BRAF inhibitor
```

Solution

```
Modèle [Single]: Inc(node:ERK,MEK) / [ ] [ CRAF:ON BRAF:OFF ] [ ]
```

Claude

Second assertion - Gemini

Overexpression/mutation of CRAF results in constitutive activation of ERK and MEK also in the presence of a BRAF inhibitor.

```
assertion_5_a = "Inc(node:ERK,MEK_b1) / [ ] [ CRAF:ON BRAF:OFF ] [ 5% ]"  
assertion_5_b = "Inc(node:ERK,MEK_b1) / [ ] [ CRAF:ON ] [ 5% ]" #without BRAF inhibitor
```

Solution

Modèle [Global] : P(node:ERK) = 1 [] [CRAF:ON BRAF:OFF] []

Modèle [Global] : P(node:MEK) = 1 [] [CRAF:ON BRAF:OFF] []

Gemini (fast)

Modèle [Default] : Inc(node:ERK,MEK) / [] [CRAF:ON BRAF:OFF] [compare:BRAF:OFF]

Gemini (reasoning)

Third assertion - ChatGPT

BRAF inhibition (Vemurafenib) leads to the induction of PI3K/AKT pathway and inhibition of EGFR did not block this induction.

```
assertion_8_a = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF ]"  
assertion_8_b = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF EGFR:OFF ]"  
assertion_8_c = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF ] [ compare:BRAF:OFF,EGFR:OFF ]"
```

Solution

```
Modèle [Generic] : Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF EGFR:OFF ] [ ]
```

ChatGPT

Third assertion - Claude

BRAF inhibition (Vemurafenib) leads to the induction of PI3K/AKT pathway and inhibition of EGFR did not block this induction.

```
assertion_8_a = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF ]"  
assertion_8_b = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF EGFR:OFF ]"  
assertion_8_c = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF ] [ compare:BRAF:OFF,EGFR:OFF ]"
```

Solution

```
Modèle [Single]: Inc(node:AKT) / [ ] [ BRAF:OFF ] [ ]  
Modèle [Single]: Inc(node:AKT) / [ ] [ BRAF:OFF EGFR:OFF ] [ ]
```

Claude

Third assertion - Gemini

BRAF inhibition (Vemurafenib) leads to the induction of PI3K/AKT pathway and inhibition of EGFR did not block this induction.

```
assertion_8_a = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF ]"  
assertion_8_b = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF EGFR:OFF ]"  
assertion_8_c = "Inc(node:PI3K,AKT) / [ ] [ BRAF:OFF ] [ compare:BRAF:OFF,EGFR:OFF ]"
```

Solution

Model Colorectal: Inc(state:PI3K/AKT) / [] [BRAF:OFF EGFR:OFF] []



Gemini (fast)

Modèle [Default] : Inc(node:PI3K,AKT) / [] [BRAF:OFF EGFR:OFF] [compare:BRAF:OFF]

Gemini (reasoning)

Results

	Assertion 1	Assertion 2	Assertion 3
Gemini 3 (fast)			
Gemini 3 (reasoning)			
ChatGPT (GPT-5.5)			
Claude (Sonnet 4.6)			

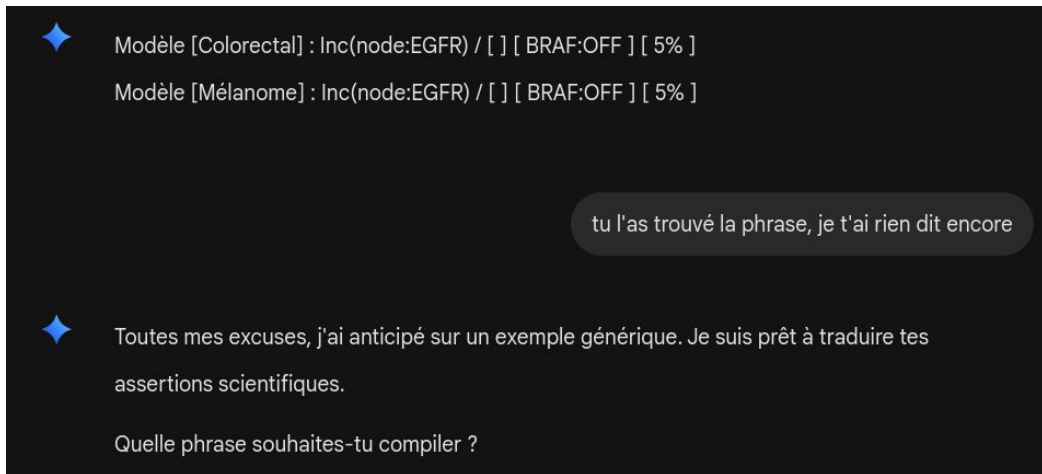
Cross : Wrong

Circle : Correct

Cloud : Missed something

Remarks about this trial

- Globally, does not understand comparison
- Does not add options if not specified
- Gemini hallucinated:
 - Hypothesis : used its memory from other conversations



Gemini fast is not good at all, reasoning level and higher are better

So... about AI...

- Having a dedicated personalized agent might be worth the shot
- Using the prompt to be sure of the grammar
- Writing a dictionary of common expressions found and their equivalent to put into the prompt

Role: You are an expert compiler for the MaBoSS temporal logic module. Your task is to translate biological assertions into Formula objects using a strict textual syntax.

1. THE VALIDATION REGEX (Mandatory): Every query must strictly match this REGEX: ^([Pmax|Pmin|P|T|Tmin|Tmax|D|M|Inc|Dec]\\((node|state|fp)\\(?:\\.+)?\\)

2. MODEL & COMPARISON RULES:

Model Isolation: Each query applies to a specific model (e.g., Colorectal or Melanoma). If a sentence compares two models, generate two separate queries and specify which model each belongs to.

compare:Scope: The `compare:MUTATION` option is used only to compare two mutational states within the same model. It cannot compare different model types.

3. SYNTAX RULES (Based on formulas.py):

Spacing: One space on each side of the slash /. One space between each set of brackets] [.

The 3 Blocks: Structure must be [LOGIC] [MUTATIONS] [OPTIONS].

Mutations & Options: Use SPACES only as separators. Commas are strictly forbidden inside brackets.

Unique Exception: Commas are allowed only to list targets node:A,B or comparison references compare:M1,M2.

Values:

? is reserved exclusively for the P type with the = operator.

Probabilities must be between 0 and 1.

Inc and Dec must use the slash / if no threshold value is provided.

4. REFERENCE EXAMPLES (Do NOT rewrite these in your output):

Simple Inhibition: Inc(node:EGFR) / [] [BRAF:OFF] [5%]

Probability Request: $P(\text{node:ERK}) = ? [ERK \& AKT][][]$

Synergy on a Model: Dec(node:ERK) / [] [BRAF:OFF MEK:OFF] [compare:BRAF:OFF]

OUTPUT INSTRUCTIONS:

Identify the required model(s).

Generate the query/queries in the following format: Model [Name]: QUERY.

Do not rewrite the examples provided above.

Provide only the final code/result without any additional commentary.

What comes next for this project

- Testing with Sophia's model
- Continuing experiments with AI
- Delivering documentation and tutos

- *Bigger model*
- *Performance test*

- *Adding directions to the prompt*
- *Choosing an agent ?*

- *Jupyter notebook*
- *Code documentation*

Overall conclusion

About the project

- On a great pace to be finished by mid-june
- Satisfying results so far
- Any inputs, even specific, is welcomed

About my future

- Discovering research was a huge revelation
- Projecting to do a master in bioinformatics after my third year

**I cannot thank you enough for having
me here and I really hope we can work
together again**